

Assignment No: 3

Student Details

Name: Tanushka Patil

PRN / Roll No: 123B1B036

Class: BTech

Division: A

Subject: DevOps

Title

Infrastructure as Code using Terraform – Provisioning an AWS EC2 Instance and a Docker Web Server

Aim

To study the concepts and workflow of Infrastructure as Code (IaC), install and configure Terraform, and develop Terraform configurations to provision an AWS EC2 instance and a local Docker web server using provider, resource, variable and output configurations.

Theory

Infrastructure as Code (IaC)

Infrastructure as Code is the practice of managing and provisioning infrastructure (servers, networks, databases, load balancers) through machine-readable configuration files instead of manual console clicks. IaC makes infrastructure repeatable, version-controlled, reviewable and easy to reproduce across environments (dev, test, prod).

- **Declarative vs Imperative:** declarative tools (Terraform, CloudFormation) describe the desired end state; imperative tools (scripts) describe the steps to reach it.
- **Benefits:** consistency, speed, automation, version control with Git, fewer human errors, and infrastructure that documents itself.
- **Idempotency:** applying the same configuration multiple times produces the same result.

Terraform

Terraform, by HashiCorp, is a cloud-agnostic IaC tool. Configurations are written in HCL (HashiCorp Configuration Language) in files ending with .tf. Terraform keeps a state file (terraform.tfstate) that maps the configuration to real resources, so it can calculate what needs to change.

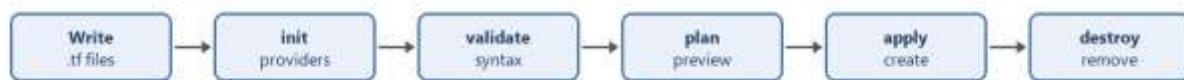
- **Provider:** a plugin that lets Terraform talk to a platform's API (aws, azure, google, docker).
- **Resource:** an infrastructure object to create, e.g. aws_instance, aws_security_group.
- **Variable:** an input parameter that makes the configuration reusable (variable blocks, terraform.tfvars, -var flag).

- **Output:** a value printed after apply, such as an instance's public IP.
- **Data source:** reads existing information, e.g. the latest AMI ID.
- **State:** the record of managed resources, stored locally or remotely (S3, HCP Terraform).

Terraform Workflow

1. Write – author the .tf configuration files.
2. terraform init – initialise the working directory and download provider plugins.
3. terraform fmt / validate – format code and check syntax.
4. terraform plan – preview the changes Terraform will make.
5. terraform apply – create or update the infrastructure.
6. terraform destroy – remove all resources managed by the configuration.

Figure: Terraform workflow



Code / Implementation

Step 1: Install Terraform

On Windows (using winget or Chocolatey), or download the binary from [developer.hashicorp.com](https://developer.hashicorp.com/terraform) and add it to PATH:

```

winget install Hashicorp.Terraform
# or
choco install terraform

terraform -version

```

Step 2: Configure AWS credentials

Create an IAM user with programmatic access (EC2 permissions), install the AWS CLI and configure it. Terraform's AWS provider reads these credentials automatically.

```

aws configure
# AWS Access Key ID      : <your-access-key>
# AWS Secret Access Key  : <your-secret-key>
# Default region name    : ap-south-1
# Default output format  : json

```

Step 3: Project structure

```

Assignment3_Terraform/
├── provider.tf      # Terraform + AWS provider configuration
├── variables.tf     # Input variables
├── main.tf          # Resources (security group, EC2 instance)
├── outputs.tf       # Output values
├── terraform.tfvars # Variable values
└── local_docker_demo/ # Same workflow using the Docker provider

```

```
|— provider.tf
|— variables.tf
|— main.tf
|— outputs.tf
```

provider.tf – Provider configuration

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

variables.tf – Variable configuration

```
variable "aws_region" {
  description = "AWS region to deploy resources in"
  type        = string
  default     = "ap-south-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}

variable "instance_name" {
  description = "Value of the Name tag for the EC2 instance"
  type        = string
  default     = "terraform-demo-ec2"
}

variable "key_name" {
  description = "Name of an existing EC2 key pair for SSH access"
  type        = string
  default     = "devops-key"
}

variable "ssh_allowed_cidr" {
  description = "Public IP allowed to SSH into the instance, in CIDR form (e.g. 203.0.113.10/32)"
  type        = string

  validation {
```

```

    condition      = can(cidrhost(var.ssh_allowed_cidr, 0))
    error_message = "ssh_allowed_cidr must be a valid CIDR block such as 203.0.113.10/32."
  }
}

```

main.tf – Resource configuration

```

# Latest Amazon Linux 2023 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

# Security group allowing SSH and HTTP
resource "aws_security_group" "web_sg" {
  name        = "terraform-web-sg"
  description = "Allow SSH and HTTP inbound traffic"

  ingress {
    description = "SSH from my IP only"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_allowed_cidr]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# EC2 instance with a simple web server
resource "aws_instance" "web" {
  ami                = data.aws_ami.amazon_linux.id
  instance_type      = var.instance_type
  key_name           = var.key_name
  vpc_security_group_ids = [aws_security_group.web_sg.id]
}

```

```

user_data = <<-EOF
#!/bin/bash
dnf install -y httpd
echo "<h1>Provisioned with Terraform</h1>" > /var/www/html/index.html
systemctl enable --now httpd
EOF

tags = {
  Name = var.instance_name
}
}

```

outputs.tf – Output configuration

```

output "instance_id" {
  description = "ID of the EC2 instance"
  value       = aws_instance.web.id
}

output "public_ip" {
  description = "Public IP address of the EC2 instance"
  value       = aws_instance.web.public_ip
}

output "public_dns" {
  description = "Public DNS name of the EC2 instance"
  value       = aws_instance.web.public_dns
}

```

terraform.tfvars – Variable values

```

aws_region      = "ap-south-1"
instance_type   = "t2.micro"
instance_name   = "terraform-demo-ec2"
key_name        = "devops-key"

# Replace with your own public IP (find it at https://checkip.amazonaws.com) followed by
# /32
ssh_allowed_cidr = "203.0.113.10/32"

```

SSH access is restricted to a single IP address through the `ssh_allowed_cidr` variable (set in `terraform.tfvars` to your own public IP followed by `/32`), while HTTP stays open so the web page can be viewed.

Step 4: Run the Terraform workflow

```

cd Assignment3_Terraform
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply -auto-approve

```

```
terraform output
terraform show

# Override a variable from the command line
terraform apply -var="instance_type=t3.micro"

# Clean up
terraform destroy -auto-approve
```

Step 5: Practical run on the local machine using the Docker provider

The same Terraform workflow was executed end-to-end on the local machine using the kreuzwerker/docker provider. This configuration pulls the nginx image and runs a web server container that serves a custom page on port 8080.

local_docker_demo/provider.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    docker = {
      source = "kreuzwerker/docker"
      version = "~> 3.0"
    }
  }
}

provider "docker" {
  host = var.docker_host
}
```

local_docker_demo/variables.tf

```
variable "docker_host" {
  description = "Docker daemon address (Docker Desktop on Windows)"
  type        = string
  default     = "npipe:////./pipe/docker_engine"
}

variable "container_name" {
  description = "Name of the web server container"
  type        = string
  default     = "terraform-web"
}

variable "external_port" {
  description = "Host port on which the web page is published"
  type        = number
  default     = 8080
}
```

local_docker_demo/main.tf

```

# Pull the nginx web server image
resource "docker_image" "nginx" {
  name      = "nginx:alpine"
  keep_locally = false
}

# Run a web server container with a custom page
resource "docker_container" "web" {
  name = var.container_name
  image = docker_image.nginx.image_id

  ports {
    internal = 80
    external = var.external_port
  }

  upload {
    file = "/usr/share/nginx/html/index.html"
    content = "<h1>Provisioned with Terraform</h1><p>DevOps Assignment 3 - Tanushka Patil</p>"
  }
}

```

local_docker_demo/outputs.tf

```

output "container_id" {
  description = "ID of the web server container"
  value      = docker_container.web.id
}

output "container_name" {
  description = "Name of the web server container"
  value      = docker_container.web.name
}

output "url" {
  description = "URL of the deployed web page"
  value      = "http://localhost:${var.external_port}"
}

cd Assignment3_Terraform\local_docker_demo
terraform init
terraform validate
terraform plan
terraform apply -auto-approve
terraform output
curl.exe http://localhost:8080
terraform destroy -auto-approve

```

Output

Output of terraform -version and terraform init (provider plugin downloaded and working directory initialised):

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform> terraform -version
Terraform v1.16.2
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.16.4. You can update by downloading from https://developer.hashicorp.com/terraform/install

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Output of terraform fmt, terraform validate and terraform providers (configuration is correctly formatted and valid):

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform> terraform fmt -check -recursive

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform> terraform validate
Success! The configuration is valid.

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform> terraform providers

Providers required by configuration:
└─ provider[registry.terraform.io/hashicorp/aws] ~> 5.0
```

Practical run with the Docker provider

terraform plan – shows the 2 resources to be created (docker_image.nginx and docker_container.web) and the outputs:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform\local_docker_demo> terraform plan

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

# docker_container.web will be created
+ resource "docker_container" "web" {
  + attach                = false
  + bridge                = (known after apply)
  + command               = (known after apply)
  + container_logs        = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  + entrypoint            = (known after apply)
  + env                   = (known after apply)
  + exit_code              = (known after apply)
  + hostname              = (known after apply)
  + id                    = (known after apply)
  + image                 = (known after apply)
  + init                  = (known after apply)
  + ipc_mode              = (known after apply)
  + log_driver            = (known after apply)
  + logs                  = false
  + memory_reservation    = 0
  + must_run              = true
  + name                  = "terraform-web"
  + network_data          = (known after apply)
  + network_mode          = "bridge"
  + read_only             = false
  + remove_volumes        = true
  + restart               = "no"
  + rm                    = false
  + runtime               = (known after apply)
  + security_opts         = (known after apply)
  + shm_size              = (known after apply)
  + start                 = true
  + stdin_open            = false
  + stop_signal           = (known after apply)
  + stop_timeout          = (known after apply)
  + tty                   = false
  + wait                  = false
  + wait_timeout          = 60
}
```

Windows PowerShell

```
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform\local_docker_demo> terraform plan (continued)
```

```
+ healthcheck (known after apply)

+ labels (known after apply)

+ ports {
  + external = 8080
  + internal = 80
  + ip       = "0.0.0.0"
  + protocol = "tcp"
}

+ upload {
  + content      = "<h1>Provisioned with Terraform</h1><p>DevOps Assignment 3 - Tanushka Patil</p>"
  + executable   = false
  + file         = "/usr/share/nginx/html/index.html"
  # (4 unchanged attributes hidden)
}

# docker_image.nginx will be created
+ resource "docker_image" "nginx" {
  + id          = (known after apply)
  + image_id    = (known after apply)
  + keep_locally = false
  + name        = "nginx:alpine"
  + repo_digest = (known after apply)
}
```

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ container_id = (known after apply)
+ container_name = "terraform-web"
+ url          = "http://localhost:8080"
```

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

terraform apply – resources created successfully:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform\local_docker_demo> terraform apply -auto
    -approve
...
Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + container_id   = (known after apply)
  + container_name = "terraform-web"
  + url            = "http://localhost:8080"
docker_image.nginx: Creating...
docker_image.nginx: Creation complete after 9s [id=sha256:df221db836e1754089190208cee7eeda94f23319705642
6eda74a43ab1abeac2nginx:alpine]
docker_container.web: Creating...
docker_container.web: Creation complete after 2s [id=aa207f104c7b389cad98c392f98076b5c0b3e4fd7a565e2b187
7694a4015335c]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

container_id = "aa207f104c7b389cad98c392f98076b5c0b3e4fd7a565e2b1877694a4015335c"
container_name = "terraform-web"
url = "http://localhost:8080"
```

terraform output – output values, the running container and the page response:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform\local_docker_demo> terraform output
container_id = "aa207f104c7b389cad98c392f98076b5c0b3e4fd7a565e2b1877694a4015335c"
container_name = "terraform-web"
url = "http://localhost:8080"

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform\local_docker_demo> docker ps --filter na
me=terraform-web

```

NAMES	IMAGE	STATUS	PORTS
terraform-web	df221db836e1	Up 1 second	0.0.0.0:8080->80/tcp

```

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform\local_docker_demo> curl.exe http://local
host:8080
<h1>Provisioned with Terraform</h1><p>DevOps Assignment 3 - Tanushka Patil</p>
```

Deployed web page opened in the browser at <http://localhost:8080>:



terraform destroy – all resources removed and no container left running:

```
Windows PowerShell
PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform\local_docker_demo> terraform destroy -au
to-approve
...
Plan: 0 to add, 0 to change, 2 to destroy.

Changes to Outputs:
  - container_id = "aa207f104c7b389cad98c392f98076b5c0b3e4fd7a565e2b1877694a4015335c" -> null
  - container_name = "terraform-web" -> null
  - url = "http://localhost:8080" -> null
docker_container.web: Destroying... [id=aa207f104c7b389cad98c392f98076b5c0b3e4fd7a565e2b1877694a4015335c]
docker_container.web: Destruction complete after 1s
docker_image.nginx: Destroying... [id=sha256:df221db836e1754089190208cee7eeda94f233197056426eda74a43ab1a
beac2nginx:alpine]
docker_image.nginx: Destruction complete after 1s

Destroy complete! Resources: 2 destroyed.

PS C:\Users\yogir\OneDrive\Desktop\Devops\Assignment3_Terraform\local_docker_demo> docker ps -a --filter
name=terraform-web
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
```

Conclusion

In this assignment, I learned about the concepts of Infrastructure as Code and the Terraform workflow (write, init, plan, apply, destroy). I installed Terraform and wrote a modular AWS configuration using a provider block, resources (a security group with SSH restricted to my IP, and an EC2 instance), input variables with validation, and outputs. I also executed the complete workflow with the Docker provider, where Terraform planned, created, reported and destroyed a web server container. This showed how infrastructure can be provisioned, changed and removed in a repeatable, version-controlled way instead of through manual console steps.

References

1. Terraform Documentation – <https://developer.hashicorp.com/terraform/docs>
2. Terraform Docker Provider (kreuzwerker/docker) –
<https://registry.terraform.io/providers/kreuzwerker/docker/latest/docs>
3. Terraform AWS Provider – <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>
4. Terraform Tutorials: Get Started – AWS –
<https://developer.hashicorp.com/terraform/tutorials/aws-get-started>
5. AWS CLI Configuration – <https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-configure.html>

Submission Links

GitHub Repository Link:

https://github.com/tanushkapatil/DevOps/tree/main/Assignment3_Terraform

Hosted / Output Link: <http://localhost:8080> (local output – removed after terraform destroy)